UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Structura sistemelor de calcul

Proiect:

Simulator de memorie virtuală

Facultate: Automatică și Calculatoare

Specializarea: CTI

An universitar: 2025 – 2026

Student: Tiucă Paul

Grupa: 30232

1. Propunere de proiect:

Proiectul de față își propune dezvoltarea unui program de simulare grafică a funcționării memoriei virtuale a unui sistem de calcul având ca obiectiv evidențierea și înțelegerea principalelor operații efectuate de aceasta prin intermediul unei reprezentări grafice. Aplicația va permite observarea modului în care memoria virtuală gestionează paginile unui process, încarca pagini din memoria fizică în cadre (frames) și înlocuiește paginile din cadre pentru a face loc pentru pagini nou.

Prin intermediul simulatorului, utilizatorul va putea urmări în timp real procese precum încărcarea paginilor în memorie, înlocuirea paginilor conform algoritmilor FIFO si LRU (Least Recently Used), cât și a actualizării tabelelor de pagini. Toate acestea vor fi accesibile cu ajutorul unei meniu interactiv care va facilita accesul la aplicație, fiind ușor de folosit și pentru persoanele deținând un nivel general de cunoștințe în domeniu.

2. Planificare

Proiectul va debuta cu etapa de definire a obiectivelor și realizarea unui studiu teoretic asupra mecanismelor de funcționare ale memoriei virtuale, incluzând concepte precum paginarea, tabelele de pagini și algoritmii de înlocuire a paginilor. În continuare, se va realiza proiectarea arhitecturii software a simulatorului, stabilindu-se structura internă, clasele principale și modul de reprezentare a memoriei fizice și virtuale.

Ulterior, se vor implementa componentele logice care simulează încărcarea paginilor in cadre fizice, gestionarea tabelelor de pagini și tratarea evenimentelor de tip page fault, împreună cu algoritmii de înlocuire a paginilor (FIFO, LRU). După finalizarea acestor module, se va trece la integrarea lor într-o interfață grafică interactivă, care va permite utilizatorului să vizualizeze în timp real procesele de alocare și înlocuire a paginilor, precum și să selecteze algoritmul dorit.

În ultima etapă, simulatorul va fi testat pe mai multe scenarii de funcționare, vor fi realizate ajustări și optimizări ale codului, iar documentația finală va fi completată cu descrierea detaliată a funcționalităților și analiza rezultatelor obținute.

3. Studiu Bibliografic

Memoria virtuală reprezintă un mecanism al sistemului de operare care oferă impresia unui spațiu de memorie continuu, mai mare decât memoria fizică disponibilă. Ea permite execuția proceselor prin maparea adreselor virtuale la adrese fizice și prin folosirea tehnicilor de paginare.

Prin paginare, spațiul de adrese este împărțit în pagini virtuale, iar memoria fizică în cadre de pagină de dimensiune egală. Acest mecanism elimină necesitatea alocării contigue și reduce fragmentarea externă.

Traducerea adreselor este realizată de unitatea de gestionare a memoriei (MMU), care folosește tabele de pagini pentru a determina cadrul fizic corespunzător fiecărei pagini virtuale. Dacă pagina solicitată nu este prezentă în memorie, se produce un page fault, iar sistemul o aduce din memoria secundară.

Când memoria fizică este plină, sistemul apelează la algoritmi de înlocuire a paginilor, precum:

- FIFO (First In, First Out) – înlocuiește pagina cea mai veche în memorie.
- LRU (Least Recently Used) – înlocuiește pagina care nu a mai fost accesată de cel mai mult timp.

Aceste concepte sunt esențiale pentru realizarea unui simulator grafic al memoriei virtuale, care permite vizualizarea proceselor de mapare, apariția page fault-urilor și aplicarea diferitelor strategii de înlocuire. Avantajele memoriei virtuale includ o gestionare eficientă a resurselor și izolarea proceselor, însă implică și un cost de performanță datorat accesului mai lent la datele stocate în memoria secundară.

4. Analiză

În sistemele reale, MMU-ul (Memory Manager Unit), TLB-ul (Translation Lookaside Buffer) și tabelul de pagini colaborează pentru a transforma adresele de memorie generate de procese în adrese fizice accesibile de hardware, menținând iluzia unui spațiu de memorie continuu și izolat pentru fiecare proces.

În cele ce urmează sunt prezentate conceptele relevante funcționării sistemelor reale, strict din perspectiva accesării paginilor de memorie – fără a detalia schimbarea de context sau alte mecanisme neincluse în proiect.

4.1. Memoria virtuală

Memoria virtuală reprezintă mecanismul prin care un proces vede un spațiu de memorie logic continuu, indiferent de dimensiunea memoriei fizice disponibile sau de fragmentarea acesteia.

Caracteristicile principale ale memoriei virtuale sunt:

- procesele au impresia că dispun de întreaga memorie a sistemului,
- memoria fizică poate fi partajată între procese,
- doar paginile necesare sunt încărcate în memoria RAM,
- protecția memoriei este asigurată prin traducerea controlată a adreselor.

Pentru a realiza această abstractizare, sistemul de operare și hardware-ul folosesc:

- divizarea memoriei în pagini virtuale de dimensiune fixă,
- un mecanism pentru a afla unde se află fiecare pagină în memoria fizică sau pe disc,
- structuri hardware specializate pentru a accelera accesul (TLB).

4.2. MMU – Memory Management Unit

MMU este componenta hardware care se ocupă de traducerea adreselor virtuale în adrese fizice.

Fluxul general într-un sistem real este:

1. Procesorul generează o adresă virtuală atunci când dorește să citească sau să scrie într-o zonă de memorie.
2. Adresa virtuală este trimisă către MMU.
3. MMU determină ce pagină virtuală este accesată.
4. Încearcă să afle imediat la ce cadru fizic corespunde pagina.

Pentru aceasta, MMU consultă mai întâi un cache specializat – TLB-ul.

4.3. TLB – Translation Lookaside Buffer

TLB-ul este o memorie foarte rapidă aflată în hardware, care memorează cele mai recente traduceri dintre paginile virtuale și cadrele fizice. Scopul său este reducerea timpului de traducere a adreselor, deoarece consultarea directă a tabelei de pagini este semnificativ mai lentă.

Funcționare generală:

1. Se caută intrarea (PID, Page Number) în TLB.
2. Dacă există:
    - se produce un TLB Hit, iar MMU obține imediat cadrul fizic,
    - accesul are cost minim.
3. Dacă nu există:
    - apare un TLB Miss,
    - MMU trebuie să consulte tabela de pagini,
    - eventual, TLB-ul este actualizat cu noua traducere.

Limitări**:**

- TLB are o capacitate redusă (zeci până la câteva sute de intrări),
- atunci când se depășește capacitatea, o intrare veche trebuie eliminată.

4.4. Tabela de pagini (Page Table)

Dacă TLB nu poate furniza traducerea necesară, MMU consultă tabela de pagini, structură care conține toate mapările paginilor virtuale către cadrele fizice.

Caracteristici ale tabelei de pagini reale:

- fiecare proces are propria tabelă de pagini,
- fiecare intrare conține:
    - numărul cadrului fizic,
    - biți de protecție (read/write/execute),
    - bitul de validare (prezentă sau nu în RAM),
    - eventual bit de acces și modificare.

Tipuri de situații:

a) Page Table Hit

Pagina se află deja în memoria fizică → se poate genera imediat adresa fizică.

b) Page Fault

Pagina nu se află în RAM, ci pe disc → este necesară încărcarea acesteia.

4.5. Page Fault

Page fault-ul este un eveniment normal într-un sistem real, declanșat atunci când o pagină virtuală cerută nu este prezentă în memoria fizică.

Fluxul general într-un sistem de operare real:

1. MMU constată lipsa paginii și generează o întrerupere.
2. Sistemul de operare:
    - decide ce pagină trebuie încărcată de pe disc,
    - dacă memoria este plină, selectează o pagină să fie înlocuită cu cea nouă.
3. Pagina nouă este încărcată în RAM.
4. Tabela de pagini este actualizată.
5. Dacă era prezentă în TLB, intrarea veche este invalidată.
6. Execuția instrucțiunii se reia.

4.6. Politici de înlocuire a paginilor

Atunci când memoria fizică este plină, sistemul are nevoie de strategii pentru a decide ce pagină existentă să fie înlocuită.
Dintre cele mai utilizate metode se regăsesc:

a) FIFO – First-In, First-Out

- Este aleasă pagina care se află de cel mai mult timp în RAM.
- Simplu de implementat, dar poate elimina pagini frecvent folosite.

b) LRU – Least Recently Used

- Este aleasă pagina care nu a mai fost utilizată de cel mai mult timp.
- Apropie cel mai bine comportamentul real al majorității programelor.
- Necesită istoric al accesărilor (timestamp-uri sau contoare).

Accesul la memorie are un cost diferit (timp) în funcție de locul în care este găsită informația: dacă această se descoperă în TLB atunci timpul necesar este de doar câteva nanosecunde sau dacă este gasită în tabelul de pagini este nevoie de zeci, eventual sute de nanosecunde pentru a fi recuperate informațiile, însă dacă pagina pe care o căutăm nu se

află în niciunul dintre aceste locuri, timpul de accesare a datelor crește la ordinul milisecundelor datorită nevoii de accesare a discului.

5. Design

Pentru a realiza o simulare corectă și modulară a unui sistem de memorie virtuală, proiectul este împărțit în componente distincte, fiecare având un rol bine definit în funcționarea aplicației. În spatele interfeței grafice, la fiecare cerere de accesare a unei pagini, sunt apelate funcții și mecanisme din modulele: MMU, TLB, Page Table, algoritmii de înlocuire a paginilor, și Controller. Această arhitectură asigură:

- separarea clară între interfața grafică și logica memoriei,
- posibilitatea extinderii ușoare (ex: adăugarea altor algoritmi),
- o simulare fidelă a comportamentului unui MMU real.

5.1 Structura generală a back-end-ului

Back-end-ul este organizat în module:

- **Controller.py** – coordonează interacțiunea dintre UI și MMU
- **business/MMU.py** – logica principală a memoriei virtuale
- **business/TLB.py** – implementează cache-ul TLB
- **business/PageTable.py** – gestionează tabela de pagini
- **business/Algorithms.py** – definește FIFO și LRU
- **models/** – conține structuri (Page, Frame, Process)

Toate aceste module comunică între ele pentru a simula corect accesarea paginilor.

5.2 Controller-ul

Controller-ul mediază între interfață și logica memoriei virtuale. El:

- primește setările din UI (număr pagini, cadre, capacitate TLB, algoritm),
- construiește un obiect MMU,
- creează un proces care conține paginile,

- oferă metodele: start_simulation() – pentru rularea automata; access_page() – pentru accesările de pagini atât pentru modul automat cât și pentru cel manual

5.3 Implementarea MMU-ului (Memory Management Unit)

MMU este „nucleul” proiectului. Acesta gestionează:

- lista cadrelor fizice
- tabela de pagini
- TLB-ul
- algoritmul de înlocuire selectat
- statisticile de rulare (timp, hits, faults)
- accesarea paginilor prin metoda: access_page()

Ordinea operațiilor este:

1) **Căutare în TLB**
    - dacă se găsește → TLB HIT
    - timpul crește cu 1 unitate
2) **Căutare în Page Table**
    - dacă există cadrul → PAGE TABLE HIT
    - timpul crește cu 10 unități
    - TLB se actualizează
3) **Page Fault**
    - timpul crește cu 100 unități
    - MMU caută:
        - un cadru liber
        - sau apelează algoritmul FIFO/LRU pentru înlocuire
    - TLB și Page Table sunt actualizate

Toate aceste informații sunt returnate sub forma unui dicționar, astfel încât UI-ul să poată colora elementele potrivite.

5.4 Implementarea TLB-ului

TLB-ul este implementat folosind OrderedDict pentru a simula comportamentul unui cache cu politică LRU:

- metoda **lookup()**:
  - verifică rapid dacă pagina este în TLB
  - dacă da, intrarea este mutată la final (LRU)
- metoda **update()**:
  - inserează o nouă intrare
  - dacă se depășește capacitatea → este eliminată prima (cea mai veche)
- metoda **invalidate()**:
  - elimină o intrare (necesară când o pagină este scoasă din memorie)

TLB-ul este complet independent de Page Table — la fel ca în sisteme reale — și este doar o optimizare.

5.5 Page Table (Tabela de Pagină)

Tabelul de pagini reține cadrul în care se află fiecare pagină.

- map_page() → asociază o pagină unui cadru
- get_frame() → verifică dacă pagina este în memorie
- unmap_page() → șterge intrarea când pagina este înlocuită

5.6 Algoritmii de înlocuire

Modulul **Algorithms.py** definește două clase:

FIFO

- înlocuiește pagina cu cel mai vechi timp de încărcare (load_time)
- simulare simplificată a cozii

LRU

- înlocuiește pagina cu **cel mai mic last_access_time**
- la fiecare hit timpul paginii este actualizat

Ambele:

- invalidează intrarea în TLB,
- șterg intrarea din Page Table,
- introduc pagina nouă în sistem,
- actualizează cadrele și tabelul de pagini.

5.7 Fluxul complet al unui acces

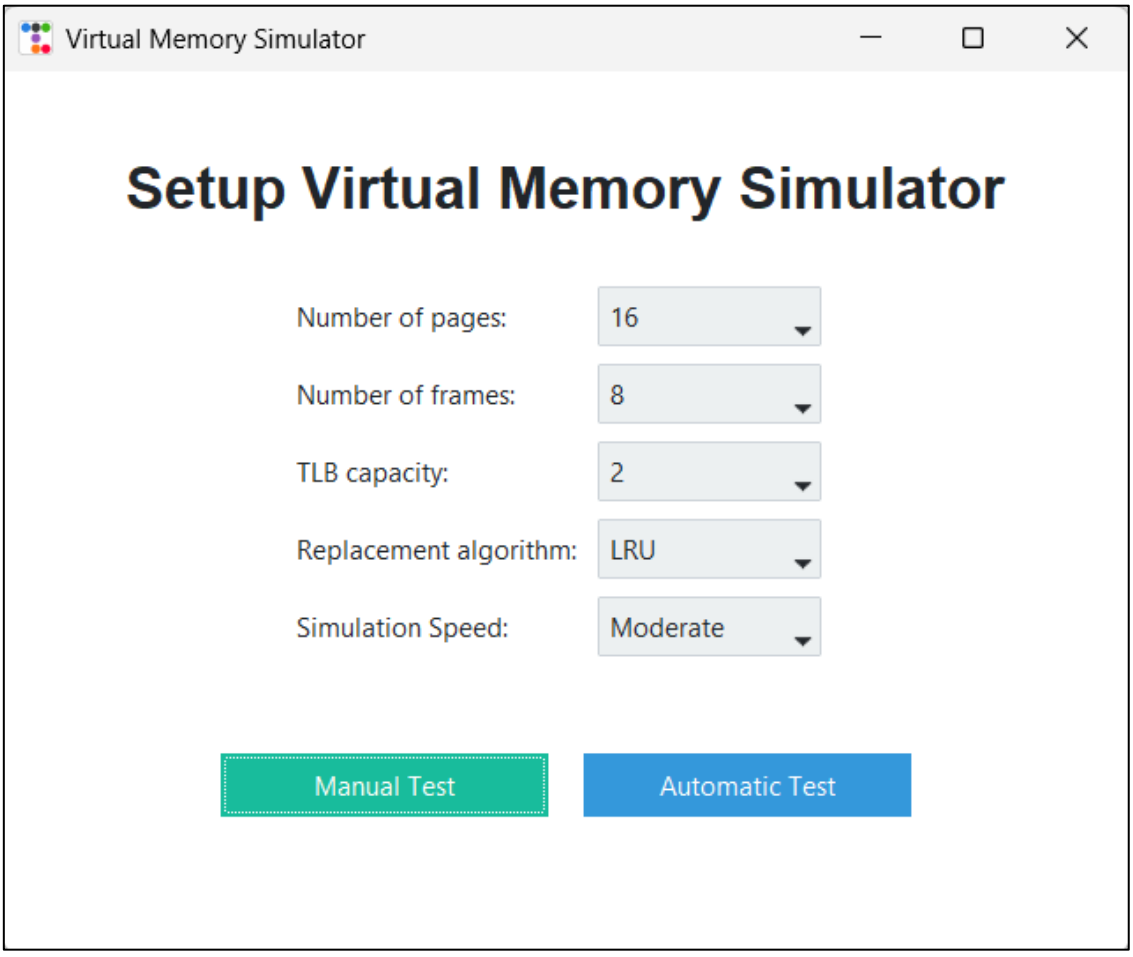
1) UI → Controller.access_page()
2) Controller → MMU.access_page()
3) MMU:
    - verifică TLB
    - verifică Page Table
    - decide page hit / fault
    - aplică LRU/FIFO la nevoie
    - actualizează:
        - timp
        - statistici
        - TLB
        - Page Table
4) MMU returnează un dicționar cu toate informațiile.
5) Controller apelează callback-ul UI-ului.
6) UI actualizează culorile și tabelele.

6. Implementare

Aplicația realizată are ca scop simularea modului de funcționare al memoriei virtuale într-un sistem de operare modern, utilizând o interfață grafică interactivă implementată în Python (Tkinter + ttkbootstrap). Sistemul permite atât simulare automată, cât și acces manual la pagini, oferind utilizatorului posibilitatea de a înțelege vizual mecanisme fundamentale precum: traducerea adreselor virtuale în adrese fizice, accesarea tabelelor de pagini, funcționarea TLB-ului, detectarea page hit-urilor și page fault-urilor, aplicarea algoritmilor de înlocuire a paginilor.

Simularea rulează secvențial, MMU-ul având rolul de a interpreta accesările paginilor și de a decide ce se întâmplă la fiecare pas. Orice eveniment produs în memoria virtuală este reflectat imediat în interfață, tabelele fiind actualizate automat cu evidențiere colorată temporară a rândurilor afectate (similar modului în care se vizualizează în sistemele reale).

Meniul principal permite alegerea numărului de pagini, dimensiunea tabelului de pagini și capacitatea TLB-ului, algoritmul de înlocuire de pagini și a vitezei simulării (pentru simularea automată).



Aplicația include două moduri de simulare:

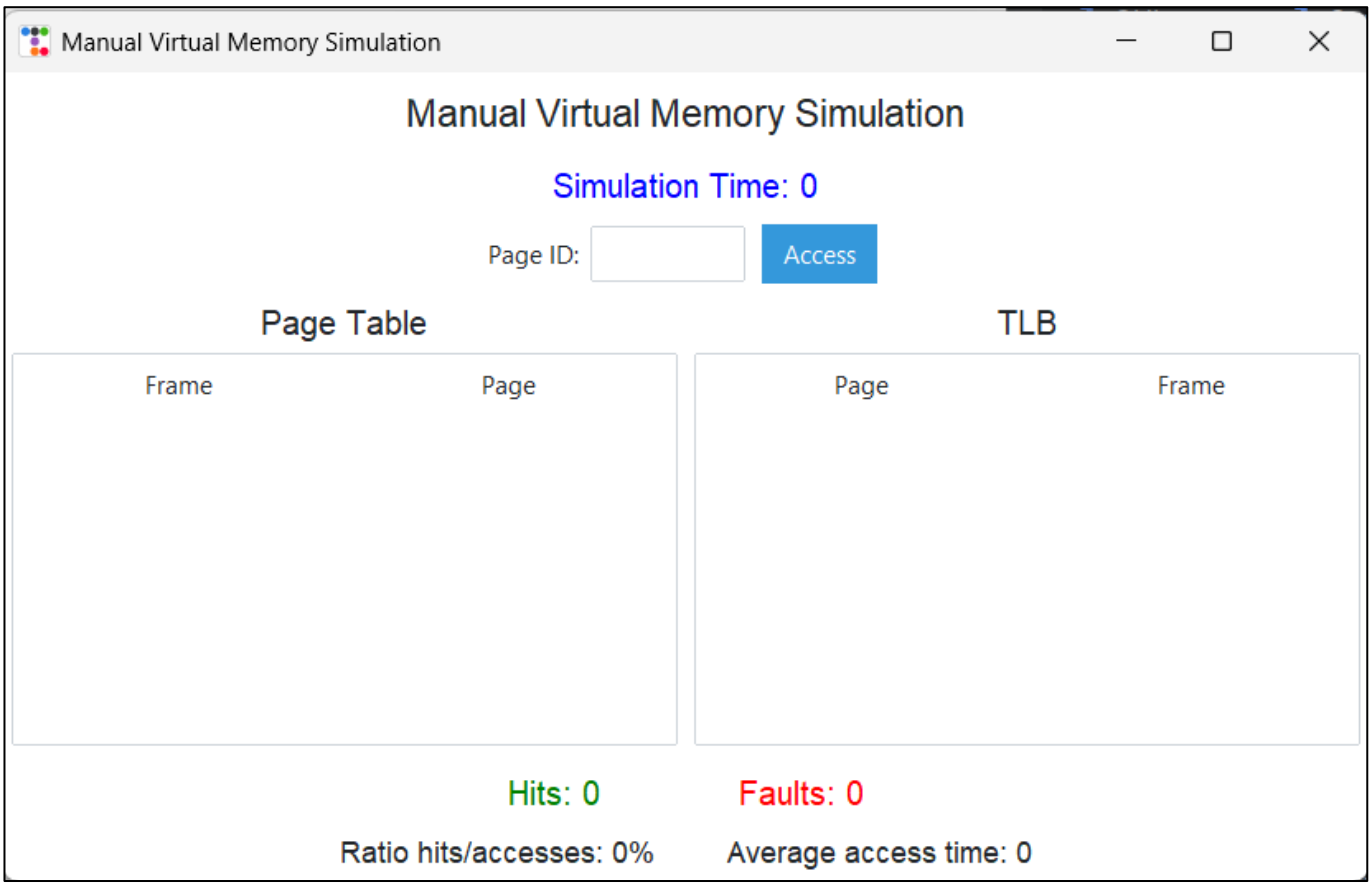
6.1. Modul manual

În modul manual utilizatorul introduce ID-ul paginii dorite, iar simularea se execută doar la apăsarea butonului ‘Access’. Acesta permite:

- rularea pas cu pas controlată,
- vizualizarea imediată a stării memoriei după fiecare acces,
- repetarea rapidă a testelor educaționale,
- analizarea individuală a fiecărei situații simulate.

În timpul simulării se pot observa urmatoarele aspecte:

1. Tabelul de pagini
2. TLB-ul
3. Timpul simulării – este incrementat cu 1 în cazul unui ‘TLB hit’, cu 10 în cazul unui ‘Page Table hit’ și cu 100 în cazul unui ‘Page fault’ pentru a simula faptul că într-un sistem real de calcul, accesarea memoriei principale este mai încet decât găsirea paginii in TLB sau în tabelul de pagini
4. Numărul de pagini găsite în TLB sau Page Table, cât și numărul de ratări
5. Procentul de “lovituri” din numărul total de accesări cât și media de timp pentru un acces de pagină

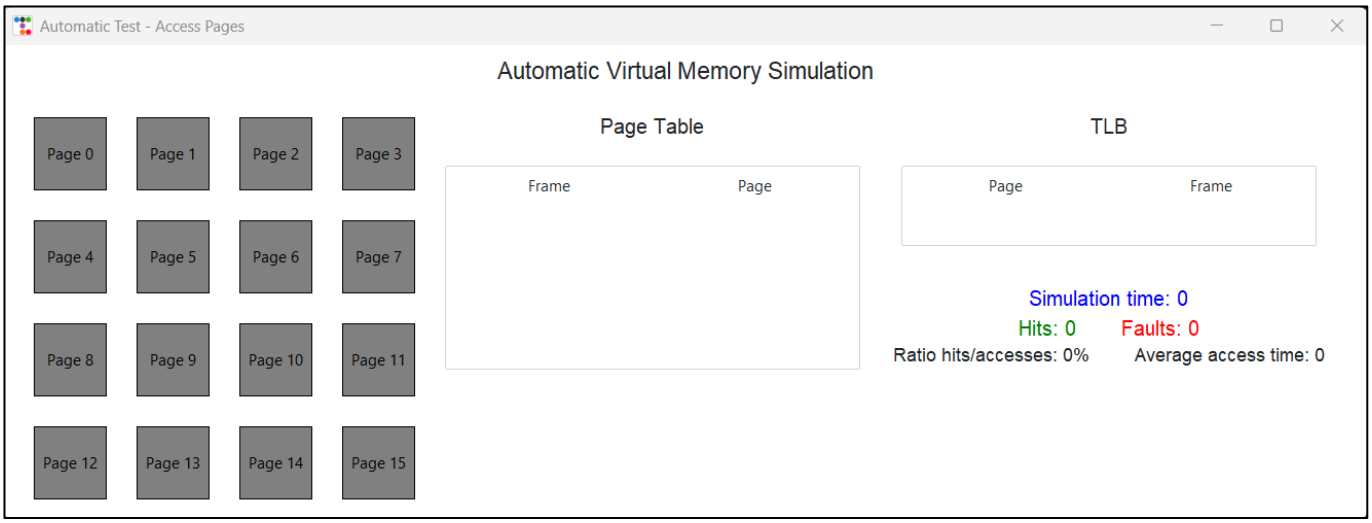


6.2. Modul automat

Oferă utilizatorului posibilitatea de a observa funcționarea sistemului de memorie virtuală fără a introduce manual paginile accesate. La deschiderea acestui mod, programul înaintează singur în timp (viteza depinde de viteza aleasă din primul meniu) și generează accesări repetate ale unor pagini virtuale, ceea ce permite analizarea vizuală a comportamentului memoriei.

În timpul simulării se pot observa:

1) Paginile unui proces – Cu numărul de pagină și cadrul în care sunt încarcate și sunt colorate astfel:
   - Gri = Pagina nu este încă încărcată în niciun cadru fizic
   - Alb = Pagina este încărcată într-un cadru fizic
   - Roșu = Pentru accesarea acestei pagini s-a produs un Page Fault
   - Oranj = Pentru accesarea acestei pagini s-a produs un Page Table Hit
   - Verde = Pentru accesarea acestei pagini s-a produs un TLB Hit
   - Albastru = Pagina a fost înlocuită în ultima accesare de pagini
2) Tabelul de pagini – Afișează cadrele de memorie fizică și ce pagini se află în acestea. După fiecare acces, tabelul se actualizează automat pentru a reflecta încărcarea unei pagini noi sau înlocuirea alteia.
3) TLB-ul – Este afișat într-un tabel separat și conține intrările cele mai recente. Pe măsură ce paginile sunt accesate, TLB-ul se actualizează automat, putând fi observate adăugările sau eliminările de intrări.
4) Evidențierea grafică a accesului – Fiecare rezultat important (TLB hit, Page Table hit, Page Fault) colorează temporar linia corespunzătoare din tabel, pentru ca utilizatorul să înțeleagă imediat ce eveniment a avut loc.
5) Statistici afișate în timp real – Sub tabele se actualizează permanent numărul de hit-uri, fault-uri, raportul procentual dintre ele și timpul mediu estimat de acces, oferind o imagine clară a performanței simulării.



6.3 Pași necesari pentru rulare

Pentru rularea aplicației este necesară instalarea și configurarea unui mediu minimal software. Proiectul este dezvoltat în limbajul Python și utilizează biblioteca ttkbootstrap pentru implementarea interfeței grafice. Pașii de instalare sunt prezentați în continuare.

1) Instalarea limbajului Python

Aplicația necesită instalarea unei versiuni Python 3.10 sau superioare.
Instalarea se poate realiza astfel:

a. Descărcarea

- Se accesează site-ul oficial: https://www.python.org/downloads/
- Se descarcă ultima versiune disponibilă pentru sistemul de operare utilizat (Windows, Linux sau macOS).

b. Instalarea

- În timpul procesului de instalare, trebuie bifată opțiunea: **"Add Python to PATH"**, pentru a permite accesarea compilatorului din linia de comandă.
- Instalarea se finalizează urmând pașii standard.

2) Instalarea managerului de pachete – pip

Utilizarea pachetelor suplimentare se face folosind pip, managerul de pachete integrat în python. Dacă pip nu este instalat, se poate activa rulând următoarea comandă în terminal: 'python -m ensurepip --upgrade'

3) Instalarea bibliotecilor suplimentare

Proiectul utilizează biblioteca ttkbootstrap (și odată cu ea, pillow), necesară pentru stilizarea interfeței grafice Tkinter. Instalarea se face prin rularea comenzii din terminal: 'pip install -r requirements.txt'

După urmarea acestor 3 pași programul poate fi rulat, rulând script-ul 'GUI.py' din pachetul 'view', din terminalul deschis în interiorul directorului principal folosind comanda: 'python -m view.GUI' sau folosind un IDE care poate rula python, precum PyCharm sau VS Code cu extensia de Python.

7. Bibliografie

- https://openosorg.github.io/openos/textbook/mm/virt-paging.html
- https://openstax.org/books/introduction-computer-science/pages/6-4-memory-management
- Ciprian Oprisa, c11_memory1, curs, UTCN, Facultatea de automatică și calculatoare, 2025
- Ciprian Oprisa, c12_memory2, curs, UTCN, Facultatea de automatică și calculatoare, 2025